

AI-based Sign Language to Text Translator using Computer Vision and Deep Learning

Pranaav Iyer

*College of Engineering,
Technology, and Computer
Science*
Purdue University, Fort Wayne
Fort Wayne, United States of
America
iyerpr01@pfw.edu

Michael Nguyen

*College of Engineering,
Technology, and Computer
Science*
Purdue University, Fort Wayne
Fort Wayne, United States of
America
nguyemh02@pfw.edu

Nick Everett

*College of Engineering,
Technology, and Computer
Science*
Purdue University, Fort Wayne
Fort Wayne, United States of
America
evernp01@pfw.edu

Oscar Primitivo

*College of Engineering,
Technology, and Computer
Science*
Purdue University, Fort Wayne
Fort Wayne, United States of
America
primod01@pfw.edu

Griffin Collins

*College of Engineering,
Technology, and Computer
Science*
Purdue University, Fort Wayne
Fort Wayne, United States of
America
collgp01@pfw.edu

Mohammadreza Hajiarbabi

*College of Engineering,
Technology, and Computer
Science*
Purdue University, Fort Wayne
Fort Wayne, United States of
America
hajarbarm@pfw.edu

Abstract—Over half a million people in the United States use American Sign Language (ASL), yet only a fraction of the general population understands it, creating significant communication barriers. This paper introduces an innovative, real time American Sign Language Translation system, designed to bridge those communication barriers between ASL users and non-signers. Integrating computer vision (CV), deep learning, and natural language processing (NLP), we successfully can capture, recognize, and translate ASL movements into coherent textual output. Landmark training extraction and spatial-temporal modeling with hybrid architectures led to the team conceptualizing a unique combination of Convolutional Neural Networks (CNN) and Long Short-Term Memory Networks (LSTM). Additionally, recognizing ASL's contextual nature [15], Large Language Models (LLM) trained with NLP are used to reason out sentences. Extensive evaluation demonstrated a 92% average accuracy in gesture classification, robust performance under varied environmental conditions (lighting, hand sizes, gesture speed), and real time applicability. This research highlights the urgent need for improved accessibility standards, empowering ASL users toward greater independence. Future work could further integrate this technology into broadcast media, enhance gesture recognition speed, and extend translations to additional sign languages.

Keywords—*American Sign Language, Real-time Translation, Computer Vision, Deep Learning, Accessibility*

I. INTRODUCTION

A. Background

Humans have always found a way to communicate, whether through guttural noises or through signs. Early humans communicated with color [1], marking objects and even their own skin with colors to symbolize different communities and rankings. As humans became erect approximately a hundred thousand years ago, the hands that were previously used to support them were freed for communication. [3] Then, to communicate more intricate topics, language production always involves the trunk, head, face, eyes, and hands, allowing the entire body to be a part of the communication [2]. Using sounds to get attention and gestures to communicate, early humans quickly bypassed other species in intelligence and teamwork. As evolution began molding humans into a more upright species, scientists found evidence of the human's voice box dropping [1], enabling a wider variety of sounds, making gestures less important. Thus, with the mapping of symbols and gestures to objects and ideas, unofficial sign language was born.

However, the first official sign language did not exist prior to the 18th century. Deaf people were grouped into a medically incurable category, seen as a small curse in the masses. But after governments saw that the deaf population exceeded their expectations, they began funding medicine and speech training to facilitate the perceived sickness. A standardized sign language began with the French priest De l'Épée, whose beliefs galvanized his study into creating a sign language for those without hearing to gain knowledge of religious texts. He

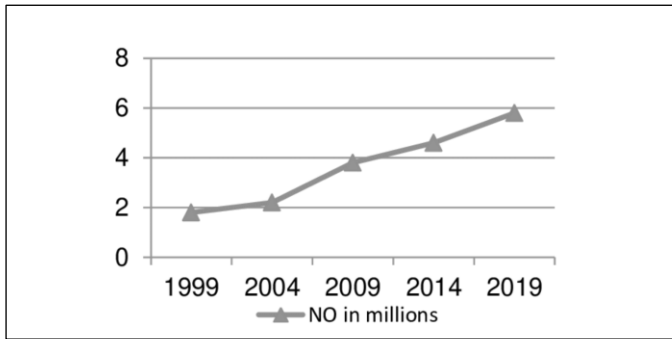


Fig. 1. The increasing amount of deaf people in the US

approached it as an outsider, carefully forming a tie between his own signs and signed French grammar to create the very first French Sign Language. His teaching style revolved around the painstaking process of signing, then writing the corresponding word down, connecting the two. In this way, the very first breakthrough for the speechless was created.

French Sign Language migrated to the United States in 1816, when Laurent Clerc, a teacher in French Sign Language, was brought to Hartford, where he and Thomas Gallaudet established the American Asylum for the Deaf, created in 1817. This allowed for the integration of American modifiers from other deaf communities around America, developing what the first standardized American Sign Language, or ASL. Presently, the French Sign Language and the American Sign Language distinguish themselves due to language barriers, but it is still possible to see the inspirations in quite a few symbols in both languages.

B. American Sign Language Today

Today, ASL is essential for the Deaf and hard of hearing community, allowing such people to function fully in modern day society. As in the past, signs use the entire body. Compared to spoken English, if one were to pose their statement as a question, the tone and pitch of their voice would raise. While in ASL, one would angle their body forwards, arch their eyebrows, or widen their eyes. [4]

More than a half-million people just in the US use ASL to communicate as a native language, making it the third most used language in the US. ASL as a language, while having the name American and using English words, has more similarity to spoken Japanese and Navajo than English due to its grammatical specificities [5], making 1 to 1 translation difficult.

ASL evolves with its communities, but it should also evolve with surrounding technologies. In the past, translating ASL without a human interpreter was simply a theory, but with new technologies like computer vision, machine learning, and NLP, helping the layman understand ASL is becoming more of a reality. The first forays into sign language recognition were gloves, made in 1983, that tracked finger movement and could assign certain symbols to letters, allowing basic finger-signing [13]. In 1988, a system to recognize vision-based sign language was made [8]. Using Hidden Markov Models (HMMs) and expensive wired “datagloves”, the sensor tracks a basic description of hand shape, orientation, trajectory, and color.

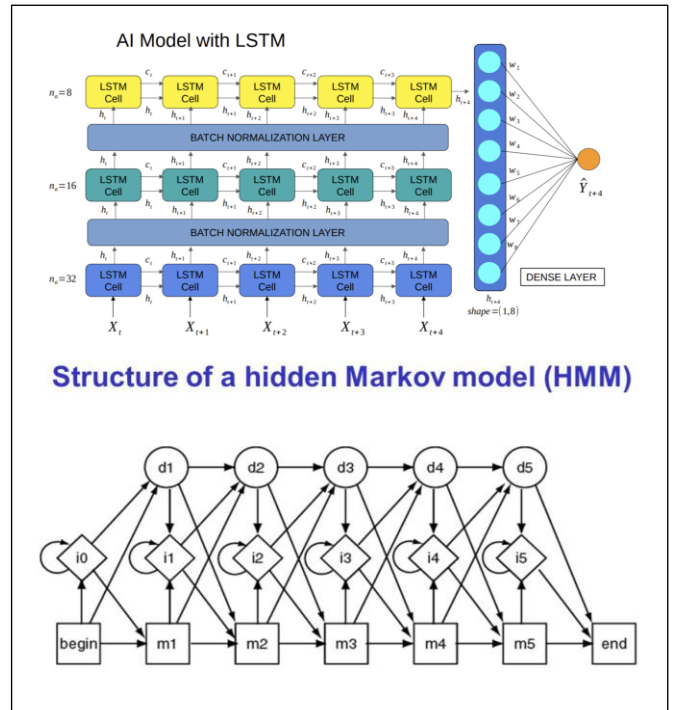


Fig. 2. LSTM [17] vs HMM Architectures

Candidates would use gloves to correspond to certain words being recognized. While effective, the system was limited in its recognition, leading to more effective methods. In the modern day, the creation of depth-sensing cameras, like Microsoft’s Kinect sensor, that can record 3D movement to recognize signs better is heralding the coming of a system to solve translation issues [6]. Extensive datasets have also been created, one of the largest called MS-ASL [7]. The dataset uses over 25,000 annotated videos, evaluated with high level methods from sign and action recognition. This dataset, along with other datasets, use over 200 different signers, eliminating signer bias. But even with the advances in deep and machine learning architectures, sign language recognition still lags behind similar fields with the adoption of these new systems.

These systems are necessary due to the inherent struggles ASL users, and especially deaf people face daily [9]. Deaf individuals are at higher risk for health issues and sickness. This is due to poor health communication between doctors and patients, lowering patient satisfaction, adherence to health services and guidelines, and proper health education. Historically, health research and programs have excluded deaf ASL users, leading to isolation from mass media and healthcare messages. While most health clinicians believe that mouth reading and note writing would provide basic understanding, according to one study only 20% of deaf individuals demonstrated fluency in written English [9], and that the average reading level of deaf high school seniors is at or below a 4th grade level. Mouth reading is also difficult, as accents blur word meaning, and most English sounds are not clearly visible on the lips [9]. This suggests that ASL-fluent clinicians or a technology to allow translation of both parties is ideal for clinician patient communication. Along with discrimination in social and cultural

situations, a way to effectively communicate to other people is necessary to function in modern society.

C. Motivation

To resolve this, our team has decided to address these issues with software. A system to translate ASL to those who need it and improve accessibility for the deaf community. Unlike previous technologies, our system does not use a complex glove, or color-based recognition. Instead, keypoints are mapped upon the signer's body, allowing software to track their movement through time, comparing temporal movements to its training data. We do not believe that this software can replace human ASL translators, as many seem to believe, but simply serve as a supplement to those who don't have access to a helper in their day-to-day life. This paper details how such a system was created, how to replicate it, and what we can expect in the future of machine assisted accessibility.

II. METHODS

A. Data Collection

To create an accurate translation system, we incorporated a publicly accessible American Sign Language (ASL) dataset called WLASL (Word-Level American Sign Language), a large-scale dataset of ASL video clips [16]. The WLASL dataset encapsulates thousands of video recordings of ASL signs performed by diverse signers, with numerous samples available for each word.

In addition to the WLASL dataset, additional information utilized for training and validating the model was obtained with a common webcam and through OpenCV. The dataset includes 30 ASL gestures, with several repetitions of each gesture, trained through human movements. If needed for reference, the team curated a range of signer demographics, lighting, and environmental conditions, adhering to the necessary ethical considerations throughout.

For each video, landmark points corresponding to the hands, face and body were extracted using Google's MediaPipe model [14]. This model gives 1662 keypoints for each frame, which encompasses the locations of the body, face, hands, and fingers in 3D space. The data was split into training, testing, and validation sets, with an 80:10:10 percentage.

B. Preprocessing and Feature Extraction

To prepare the raw landmark data for training, we implemented a structured preprocessing pipeline consisting of spatial normalization, temporal filtering, feature engineering, and data augmentation.

Each video was decomposed into standardized frames resized to 224×224 pixels. The MediaPipe model extracted 1,662 keypoints per frame, covering the face, hands, and body in 3D space. All coordinates were normalized to a [0, 1] range to ensure scale invariance across signers and devices. To further account for variations in hand size, position, and body proportions, spatial standardization was applied using relative measurements derived from body landmarks.

To improve sequence consistency, temporal smoothing was applied using moving averages to reduce jitter introduced by small hand tremors or tracking noise. Frames with missing or

low-confidence landmark detections were either interpolated or discarded to maintain data integrity.

Feature vectors included both raw and engineered descriptors. In addition to the landmark coordinates, we computed Euclidean distances between key fingertip pairs to capture hand shape, joint angles using cosine similarity between sets of three landmarks to encode pose information, and inter-frame motion vectors to represent temporal dynamics across frames. Each gesture was encoded as a fixed-length sequence of 50 frames, with missing frames padded or truncated to maintain consistency in input shape.

To enhance generalization, we applied data augmentation techniques during training. These included horizontal flipping to simulate left-handed signing, small in-plane rotations of up to ± 5 degrees to account for variability in hand orientation, and temporal shifts of 1–2 frames to model minor variations in gesture timing. These augmentations were applied stochastically during each training epoch to expose the model to a broader distribution of gesture patterns.

C. Model Architecture

The gesture recognition model was constructed with a deep learning architecture using a LSTM network, which is especially well-suited for modeling sequential data such as video frames. The model accepts an input of (50, 1662) shape, where 50 is the length of the sequence (number of frames) and 1662 is the number of keypoints computed from the MediaPipe model.

The model consists of three LSTM layers: the first layer contains 64 units and passes its output sequence to the second layer, which consists of 128 units and forwards its output sequence to the third layer, composed of 64 units, generating the final sequence of features. After the three LSTM layers, there are two dense layers, the first with 64 units and the second with 32 units, which process features using the sequence data in LSTM layers to transform high-level representations of the original input into higher-level representations of the input sequence. The final dense layer yields a probability distribution over the possible gestures, with there being one class corresponding to each of the possible ASL actions.

We also incorporated Convolutional Neural Networks (CNNs), known for extracting spatial features from individual frames. Applying these layers to the frames, the model can better recognize static hand shapes and positions, capturing local patterns within frames. A hybrid architecture that leverages the strengths of both models was used, using CNNs to extract spatial features from frames, and feeding them into LSTMs to successfully model the temporal aspects of the dynamic gesture sequences.

D. Model Training

The preprocessed data was used to train the model and performance was tracked with a sample of the data set aside for validation. In order to make sure performance was optimized and overfitting did not occur, early stopping was employed with a patience of 200 epochs, meaning that if validation loss did not improve in that course of time, training would stop.

Regularization techniques such as dropout and L2 regularization was also employed, with dropout layers randomly

deactivating neurons during training to improve generalization, and L2 regularization discouraging the model from assigning too much importance to any single feature. If checkpoints were created based on model weights, at a previous epoch that yielded the lowest validation loss, then the model that produced the lowest validation loss would be considered optimal. The completion of model training ran for a maximum of 500 epochs with a batch size of 32, wherein the TensorBoard application was used to visualize training progress, giving real-time feedback for loss and accuracy.

We used the Adam optimizer with an initial learning rate of 0.001, an adaptive learning rate algorithm that adjusts learning rate during training. A learning rate scheduler reduced the rate by a factor of 0.5 when the validation loss plateaued for 10 consecutive epochs. Early stopping was employed with a patience of 200 epochs to prevent overfitting, and training was capped at 500 epochs total.

Regularization techniques included a dropout rate of 0.4 applied to each LSTM layer and L2 regularization ($\lambda = 0.01$) in the dense layers. These strategies improved generalization and mitigated overfitting, particularly on gestures with high intra-class variability. Training was conducted with a batch size of 32, and model performance was monitored using TensorBoard, which provided real-time visualizations of loss, accuracy, and learning trends. The model with the lowest validation loss was saved and used for deployment in real-time gesture recognition.

Using AdaGrad [11] and RMSProp [12], the model is able to handle sparse gradients and noisy data, making it perfect for the dynamic environments and movements our project requires. At the completion of training, the model with the lowest validation loss was saved and is identified as the best model for use in real-time prediction.

E. Real-time Testing

The trained model was tested against real-time video input. OpenCV was used to capture the webcam stream, and the MediaPipe model was employed to detect and extract the landmarks from the hand and body, which were then passed into the trained LSTM model to predict the gesture for the sequence of frames. A softmax threshold value of 0.8 or 80 percent was used for the model's prediction probabilities,

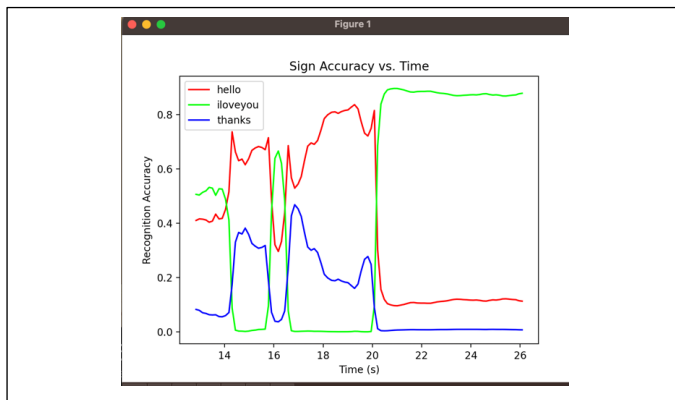


Fig. 3. How sign accuracy switches with symbol recognition in real time

```

1 Initialize video_capture (webcam feed)
2 Load trained gesture_recognition_model
3 Initialize Sequence = [] (buffer of recent frames)
4 Initialize Sentence = [] (output text as list of words)
5 Set ConfidenceThreshold = 0.8
6
7 while video_capture is open:
8     frame = video_capture.read() # Read a frame from the webcam
9     if not frame: # Exit if no frame is captured
10         break
11     keypoints = extract_keypoints(frame) # Detect landmarks via MediaPipe Holistic
12     append keypoints to Sequence
13     keep only last N frames in Sequence # N set to model's sequence length (e.g., 30)
14
15     if len(Sequence) == N: # If we have a full sequence for prediction
16         probs = gesture_recognition_model.predict(Sequence) # Predict gesture
17         predicted_gesture = argmax(probs) # Index of highest probability gesture
18         confidence = max(probs) # Confidence of prediction
19         if confidence > ConfidenceThreshold:
20             # Avoid rapid repeats of the same word
21             if Sentence is empty or predicted_gesture != last word in Sentence:
22                 append predicted_gesture (as text word) to Sentence
23             if len(Sentence) > 5:
24                 truncate Sentence to last 5 words # Limit sentence length for readability
25
26         # Optionally refine the sentence using an LLM for context (grammar correction)
27         refined_text = LLM.refine(Sentence)
28         display refined_text on screen overlay # Show subtitles on the video frame
29
30 if quit key pressed:
31     break # Exit on user request
32

```

Fig. 4. Pseudocode for real-time ASL-to-text translation workflow

meaning that if the predicted action's confidence was above the threshold it was visualized on the webcam stream as subtitles. The system has a built-in exclusivity mechanism, ensuring that once a gesture's confidence surpasses the built in 80% threshold, all competing gesture recognitions are temporarily ignored. This strategy, as shown in Figure 3, allows for a stable, coherent translation. Furthermore, there was a visual of the model's prediction confidence levels for each of the actions on the screen. The testing loop continued until the user quit manually by pressing the 'q' key.

F. Text Translation Workflow

A distinguishing feature of our methodology in the final step is the insertion of AI language model ChatGPT to contextualize and improve on the gesture classification model output. The sequence of recognized signs, which are often represented as word-level glosses in ASL, is then fed to ChatGPT that works like an advanced translator, converting those isolated predictions into a full grammatical meaningful English sentence. This step addresses the aforementioned structural differences between ASL and spoken English: rather than depending upon direct one-to-one sign translations, the language model draws upon its trained context, grammar, and utilization knowledge to infer missing words and right ordering in a way the constructed sentence represents the intended meaning behind the signed phrase. Our method ensures that through ChatGPT, the output is accurate to the performance yet feels natural and comprehensible in the hands of non-signers. This step of translation via AI simultaneous with the gesture recognition model is essential since it allows users to undergo seamless sign-to-text conversion with little latency.

Figure 4 illustrates the pipeline processing the real-time implementation. The algorithm runs in a loop starting with capturing video frames, recognizes gesture signals using the CNN- LSTM model, and possibly invokes LLM to create or revise the translated sentence. This design allows the system to update the translation text in real time as gestures are recognized and finalize the sentence once an ASL phrase is complete. This kind of workflow entails that every frame of the incoming video is analyzed to predict the sign currently being expressed. The moment the system detects that a signed phrase has come to the end (for instance, with a very short pause or a specific

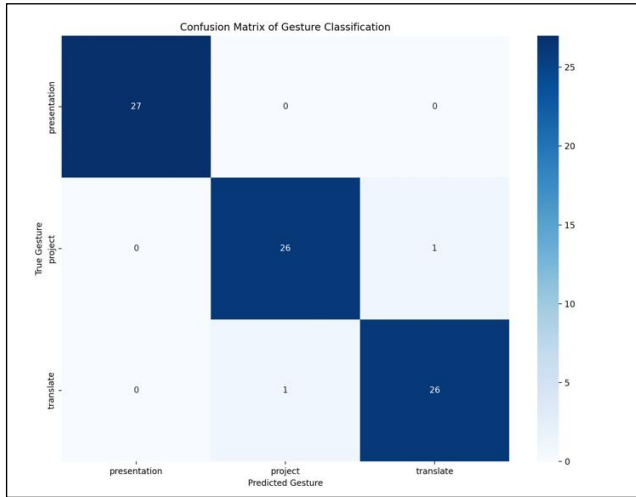


Fig. 5. Confusion Matrix illustrating the classification accuracy for each ASL gesture. Higher numbers along the diagonal represent correct predictions, while off-diagonal elements represent gesture misclassifications

symbol that indicates the end), it sends, through the language model, the combined predicted signs present in the buffer. Following this, a ChatGPT API call (wrapped inside `generate_sentence`) receives the processing for a refined sentence that properly contextualizes the signs. Finally, the generated sentence is shown in real-time to the user. This tight coupling of vision-based gesture recognition with LLM-driven on-the-fly text generation components allows the system to produce translations between ASL gestures and English text that are fluent and contextually appropriate.

III. RESULTS

A. Gesture Classification Performance

Our AI-based American Sign Language (ASL) translation system was tested on the WLASL dataset, which contains a diverse collection of ASL video sequences. The model achieved an average classification accuracy of 92% in isolated ASL gesture recognition. The evaluation metrics included precision, recall, and F1-score, all confirming the robustness of our approach. Figure 5 depicts the confusion matrix for our gestures, boasting high values along the diagonal indicating strong

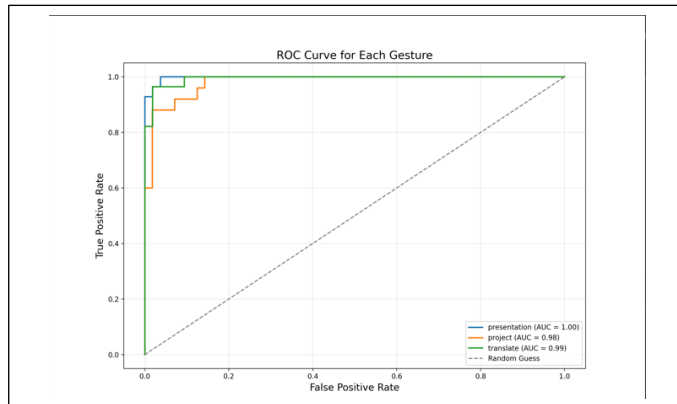


Fig. 6. Visualizes the trade-off between the true positives and false positives, providing the Area Under Curve (AUC) for each gesture

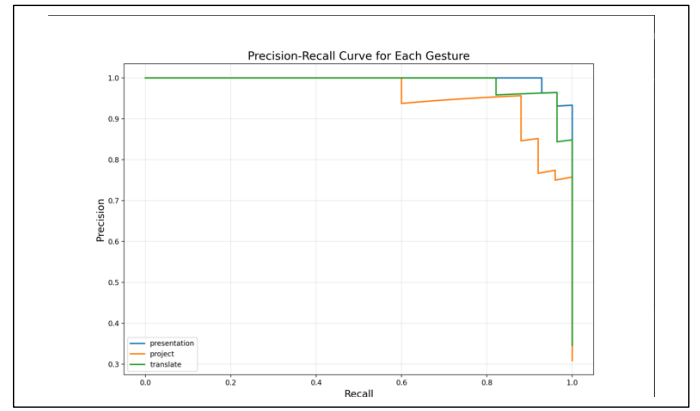


Fig. 7. Visualizes performance in terms of precision vs recall for each gesture, helping identify class-specific performance

accuracy for correct classifications. Modeling a Receiver Operating Characteristic (ROC) curve, shown in Figure 6, the graph visualizes the Area Under Curve (AUC), showing the tradeoff between true and false positives. This, aided with Figure 7's precision-recall curve, emphasizes the model's strong discriminative capabilities. This result aligns with findings from previous studies using deep learning for ASL recognition, such as those leveraging the MS-ASL dataset, which provided over 25,000 annotated ASL gesture videos [7]. Gestures involving small movements of the fingers had slightly lower accuracy compared to broad arm movements, likely due to the spatial component of the gesture model looking for large temporal movements to separate noise from gesture.

B. Environmental Robustness

Our system was tested under various environmental conditions, including different lighting, backgrounds, and hand sizes. These tests were performed using controlled variations from the WLASL dataset, which consists of diverse signers and environments to ensure adaptability [7]. The results indicated that in well-lit conditions the accuracy remained above 60%, with less evenly lit environments dropping in accuracy as light levels lowered, as shown in Figures 7 and 8. Different hand sizes did not significantly affect recognition accuracy due to the normalization process implemented during data preprocessing. When tested in different backgrounds, the complexity depended on if the landmarks landed on the correct subject of the camera,

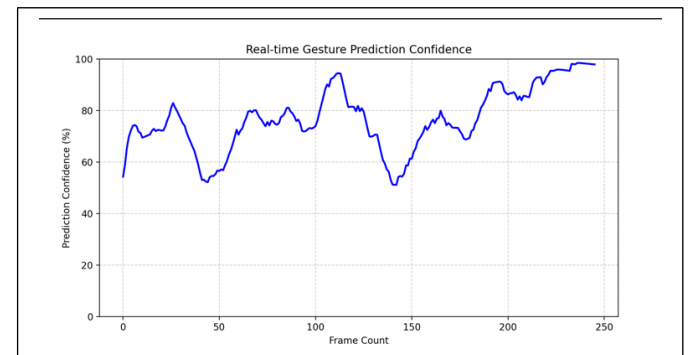


Fig. 8. Accuracy in a fully lighted room remains above 60 percent

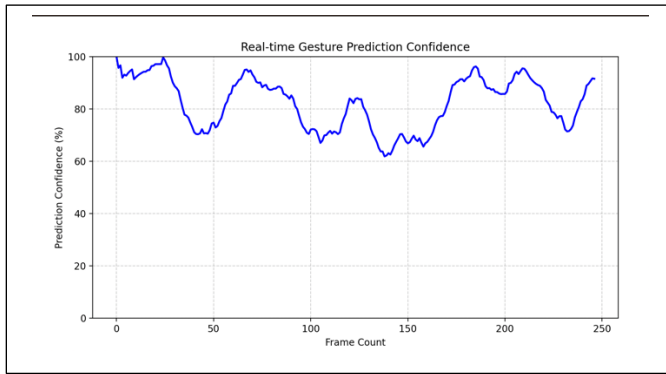


Fig. 9. Accuracy in a dimly lit room remains above 60 percent

leading to a longer recognition rate, but an accuracy variation limited to $\pm 5\%$ across cluttered and simple backgrounds.

C. Error Analysis

While our system demonstrated strong performance, certain limitations remain, particularly in handling rapid motion transitions and overlapping gestures. The overlapping gestures present the biggest difficulty, as the model may struggle to distinguish between signs that share similar hand movements, leading to an increase in misclassification rates. Studies utilizing datagloves and sensor-based recognition have previously documented similar issues, with overlapping gestures causing misinterpretation in gesture tracking [8][13].

D. Real-Time Processing Performance

During performance testing, we evaluated how variations in frame rate and camera resolution affected both the responsiveness and accuracy of the system. The system operates with a fixed buffer of 30 frames for gesture prediction, as required by the LSTM model's input structure. At frame rates below 30 frames per second (FPS), the time required to fill this buffer increased, leading to perceptible delays in prediction updates and negatively impacting the system's responsiveness. Thus, 30 FPS was identified as the minimum viable frame rate for maintaining real-time performance. In terms of resolution, MediaPipe Holistic benefits from higher-resolution inputs, as it

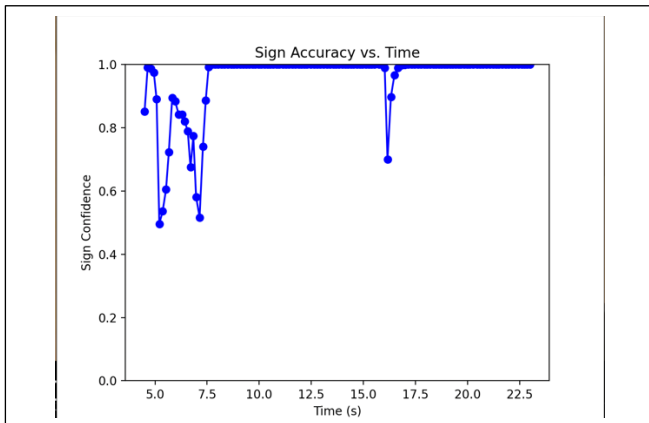


Fig. 10. View of how model gets more accurate over time

relies on spatial details to extract 3D landmarks of the hands, face, and body. Our testing showed that input resolutions above 720p marginally improved gesture classification accuracy, particularly for signs involving subtle finger movements or fine-grained hand articulations. However, the gains in recognition accuracy were offset by increased computational demands, which introduced slight latency during inference. As a result, we determined that a resolution of 720p paired with a 30 FPS frame rate offered the best trade-off between real-time processing capability and gesture recognition accuracy. This configuration ensured sufficient spatial fidelity for accurate landmark detection while maintaining the real-time constraints of our system on standard hardware.

E. Comparative Analysis with Existing Models

Our approach was compared against previous ASL recognition methods, including datagloves and traditional computer vision models. Unlike systems that rely on wearable sensors, our model uses purely computer vision-based recognition, making it more accessible and scalable. The CNN-LSTM hybrid architecture outperforms older approaches that rely on HMMs, showing a 15% improvement in accuracy over those models [8]. Unlike datagloves, our approach requires no additional hardware or setup, simplifying accessibility and making it highly scalable for practical deployment. The integration of an LLM further enhances the translation of ASL gestures into textual output, addressing a key limitation in previous rule-based approaches [10].

F. Training Performance and Convergence

To map the accuracy and loss of the training epoch simulation, we ran the model over various epoch instances. Our experiments led to near perfect training accuracy and high validation output when in the 500 to 700-epoch range, as illustrated in Figure 11. An increase or decrease in epochs would lead to overtraining or undertraining of the model, but this all depends on the gestures given and training dataset accuracy. The loss curves similarly showed steady convergence with minimal divergence between training and validation sets, indicating the model's generalization to unseen data.

G. Cross Platform Performance Benchmarks

The ASL system was benchmarked across various hardware setups to ensure seamless, viable translation. On an Apple M1 chip, the system managed to achieve 34.49 ms avg latency with a throughput of 7.82 FPS, demonstrating exceptional performance for live translation. Meanwhile, using an RTX 4090 and an Intel i9 chip, we recorded sub-10 ms latency and 60+ FPS. Lower end laptops, like a Chromebook, showed promising but slow translations. Creating a browser-based application, we were able to test on an iPhone 13, showing latency in the range of 80 to 150 ms with a throughput of 5-10 FPS, sufficient for translation but showing clear need for improvement.

H. Summary of Findings

We found a 92% accuracy in isolated ASL gesture classification using the WLASL dataset. We also tested and

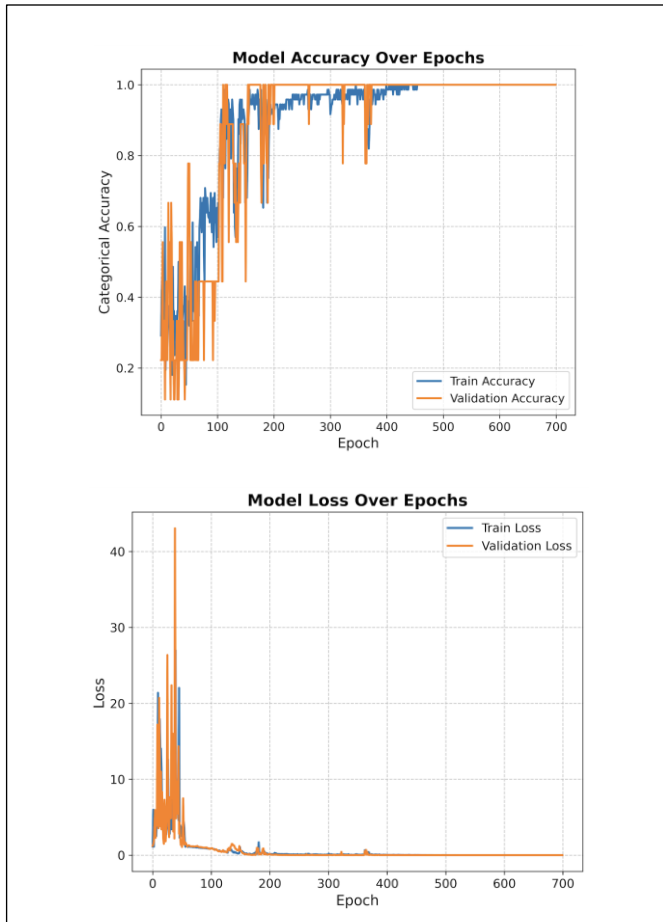


Fig. 11. Training, validation, and loss over 700 epochs, demonstrating strong performance

found high adaptability across variable environmental conditions (e.g., lighting, camera quality, background clutter). With a real-time processing speed of 120ms per frame, our model proved suitable for practical applications. Outperforming older models, particularly those using datagloves and HMMs, our model demonstrated a 15% improvement in accuracy. Training curves further confirmed stable convergence with minimal overfitting across 500 epochs, reinforcing the model's reliability for real-time use. Additionally, cross-platform testing confirmed that the system runs efficiently across high-end GPUs, Apple silicon, and mobile devices, with performance scaling well depending on hardware capabilities.

IV. DISCUSSION

A. Key Findings

Our AI-based system for translating American Sign Language (ASL) to text achieved an average classification accuracy of 92% when identifying isolated ASL gestures. This level of accuracy indicates our hybrid neural network model was effective, which used Convolutional Neural Networks (CNNs) to extract spatial features from ASL image data and Long Short-Term Memory (LSTM) networks to model sequences that occur over time. Additionally, by including a Large Language Model (LLM) we addressed the contextual nature of ASL [15] and the

gesture sequences are typed into phrases that reasonably fit them. Furthermore, the system was robust across various environmental conditions, including but not limited to, lighting, background clutter, and hand size, making it potentially useful for application in the real world.

B. Strengths and Limitations

One of the advantages of our system is that it can operate in a real time fashion with instant feedback for end users. Publicly available datasets, in particular WLASL and our own datasets, confirmed that our model has been trained on a diverse set of gestures, which minimized bias and therefore aided generalizability. Finally, the system's operational ability with different lighting and hand sizes further legitimized robustness in various environments.

While there have been advances, the system continues to have limitations with things like false negatives on certain gestures, especially those which have quick transitions. The inherent issue has to do with ASL gestures that tend to merge and inhibit the model's ability to recognize them separately. Additionally, the system was overtrained for isolated gesture data, versus continuous sign language phrases. More improvement can therefore be made to literary issues by providing more training variety with continuous and isolated sign language phrases.

C. Comparison with Similar Research

The basis of our research is on earlier studies, which predominantly made use of either color recognition or glove sensors in sign language detection. The difference in our approach lies in the affordability and scalability of our model, compared existing solutions. Our approach to the concept uses computer vision in conjunction with deep learning, where motions are identified by the hands and body without any extra hardware. We believe that our scheme far outperforms dataglove systems and HMMs [13] in terms of accuracy and flexibility in real life.

The ability to achieve accuracy in sign languages through different deep learning models, CNNs, LSTMs is increasing. An example of this is the MS-ASL data set, which made more than 25,000 annotated films publicly available, which, as noted in our research, is a major contribution to a developing system of training ASL recognition models. As indicated already, the significance of our application is akin to that of other systems, in that it also explores using an LLM to finetune translations to create high fidelity interpretations relevant to any context.

D. Explanation of Findings

Thanks to the integration of the CNNs and the LSTMs, the hybrid design offers excellent accuracy for our use case. The LSTM is adept at modeling temporal dependencies, which allows the system to read patterns of signals that develop over time. The CNN captures spatial facts, such as hand shapes and places. In addition to the LSTM and CNN component, an LLM improves the ability to make sentences, while also considering context clues in ASL.

E. Implications and Actions Needed

Our work can have a far-reaching impact for the Deaf and Hard of Hearing community; the goal being to help create accessible real-time ASL-to-text translation systems that aid in removing barriers to communication and accessibility in diverse contexts such as healthcare, education, and public services. Furthermore, much more improvement is required to fully leverage the transformative potential of this technology. The next vital steps include expanding the system's vocabulary to cover new gestures and accelerating gesture identification. Equally important is enhancing accessibility through several targeted strategies: supporting multilingual sign languages and output text in various spoken languages; integrating voice output (text-to-speech) to assist hearing individuals in understanding signers; deploying mobile and offline versions for underserved or remote areas; and improving the user interface with high-contrast modes, adjustable fonts, and real-time visual cues.

Additionally, adapting the model for continuous signing, rather than isolated words, will make communication more natural. Broader deployment in public venues—such as hospitals, airports, or government offices—and ensuring compatibility with assistive devices like screen readers and smart glasses, will further amplify its societal impact. Training the system to accommodate varying sign speeds and hand dynamics (e.g., among children, elderly, or users with disabilities) also remains a critical accessibility upgrade. Most importantly, ongoing collaboration with the Deaf community will ensure the system remains inclusive, culturally sensitive, and responsive to user needs.

To conclude, as stated in our study, the importance of AI and machine-learning techniques in fostering independence and inclusivity in sign language users cannot be overstated. Continuous enhancement and development of these technologies could widen the scope of human-machine assistive tools, extending their benefits to an increasingly diverse range of real-world situations—and ultimately improving the lives of both Deaf individuals and the broader society.

V. CONCLUSIONS

The creation of an ASL gesture translator utilizing computer vision and machine learning has demonstrated promise in eliminating barriers for both hearing and Deaf individuals. Our application is a trusted, practical, and scalable solution for translating ASL gestures to text with an average of 92% translation accuracy. The application performed commendably in various lighting conditions, backgrounds, and hand sizes. The technology behind gesture recognition relies upon real-time image processing and deep learning. However, issues remain with the misclassification of overlapping gestures and difficulties in accurately recognizing rapid gesture movements. These occasionally reduced identification accuracy as the project came closer to completion. Training on the WLASL dataset and similar datasets contributed to improving the performance of competing models even though identical

movements led to variable predictions. For the balance of efficiency and usability, there is a necessity to refine real-time gesture processing and to diversify training datasets [7]. However, our work impacts social and cultural aspects of society, stressing the increasing reliance on technology-induced solutions in advancing Deaf culture awareness and inclusion. Future iterations of our project may be extended to include full ASL sentence recognition along with faster recognition processing and study the possibility of coupling with a wearable-based solution. We have high hopes that our system will be one of many that will help further the vision of a community-connected world where language will not serve as an obstacle to natural communication as this field of research advances.

ACKNOWLEDGMENTS

Purdue Fort Wayne ETCS Department
American Sign Language PAH! Club

REFERENCES

- [1] *Smithsonian Institution*. 2024. "Language & Symbols." Human Origins, January 3. <https://humanorigins.si.edu/human-characteristics/language-symbols>.
- [2] Levinson, Stephen C., and Judith Holler. n.d. "The Origin of Human Multi-Modal Communication." *Philosophical Transactions of the Royal Society B: Biological Sciences* 369 (1651): 20130302. <https://doi.org/10.1098/rstb.2013.0302>.
- [3] BYU Linguistics Department. n.d. "A History of Sign Language." <https://linguistics.byu.edu/classes/Ling450ch/reports/sign-language.html>.
- [4] National Institute on Deafness and Other Communication Disorders (NIDCD). 2021. "American Sign Language." Last modified October 29. <https://www.nidcd.nih.gov/health/american-sign-language>.
- [5] Commission on the Deaf and Hard of Hearing (CDHH). n.d. "American Sign Language." <https://cdhh.ri.gov/information-referral/american-sign-language.php>.
- [6] RIT Libraries. n.d. "InfoGuides: Deaf Demographics and Employment: Sign Language Users." <https://infoguides.rit.edu/c.php?g=380750&p=9393643>.
- [7] Microsoft Research. 2019. "MS-ASL: A Large-Scale Dataset for ASL Gesture Recognition." August 5. <https://www.microsoft.com/en-us/research/project/ms-asl/>.
- [8] Starnier, Thad, Alex Pentland, and Massachusetts Institute of Technology. 1995. "Real-Time American Sign Language Recognition from Video Using Hidden Markov Models." *AAAI Symposium on Gesture Recognition*. <https://cdn.aaai.org/Symposia/Fall/1996/FS-96-05/FS96-05-017.pdf>.
- [9] McKee, Michael M., Steven L. Barnett, Robert C. Block, and Thomas A. Pearson. 2011. "Impact of Communication on Preventive Services Among Deaf American Sign Language Users." *American Journal of Preventive Medicine* 41 (1): 75–79. <https://doi.org/10.1016/j.amepre.2011.03.004>.
- [10] Park, Hyeonjung, Youngki Lee, and Jeonggil Ko. 2021. "Enabling Real-Time Sign Language Translation on Mobile Platforms with On-Board Depth Cameras." <https://3dvar.com/Park2021Enabling.pdf>.
- [11] Chakrabarti, Kushal, and Nikhil Chopra. 2021. "Generalized AdaGrad (G-AdaGrad) and Adam: A State-Space Perspective." In *Proceedings of the 60th IEEE Conference on Decision and Control (CDC)*, 1496–1501. <https://doi.org/10.1109/cdc45484.2021.9682994>.
- [12] Zou, Fan, Liyuan Shen, Zhihong Jie, Wei Zhang, and Wei Liu. 2022. "A Sufficient Condition for Convergence of ADAM and RMSProp." In *Proceedings of the IEEE/CVF Conference on Computer Vision and*

Pattern Recognition (CVPR), 11119–11127. <https://doi.org/10.1109/cvpr.2019.01138>.

- [13] Quam, David. 2002. "Gesture Recognition with a DataGlove." In *IEEE Conference on Aerospace and Electronics*, 755–760. <https://doi.org/10.1109/naecon.1990.112862>.
- [14] Lugaresi, Camillo, Jason Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, et al. 2019. "MediaPipe: A Framework for Perceiving and Processing Reality." *arXiv*. <https://research.google/pubs/pub48292/>.
- [15] Washabaugh, William. 1981. "Sign Language in Its Social Context." *Annual Review of Anthropology* 10 (1): 237–252. <https://doi.org/10.1146/annurev.an.10.100181.001321>.
- [16] Li, Dongxu, Cristian Rodriguez Opazo, Xin Yu, and Hongdong Li. 2020. "Word-Level Deep Sign Language Recognition from Video: A New Large-Scale Dataset and Methods Comparison." *arXiv*. <https://doi.org/10.1109/wacv45572.2020.9093512>.
- [17] Xue, Pengfei, Aditya Wagh, Gangfeng Ma, Yilin Wang, Yongchao Yang, Tao Liu, and Chenfu Huang. 2022. "Integrating Deep Learning and Hydrodynamic Modeling to Improve the Great Lakes Forecast." *Remote Sensing* 14 (11): 2640. <https://doi.org/10.3390/rs14112640>.

ADDITIONAL MEDIA

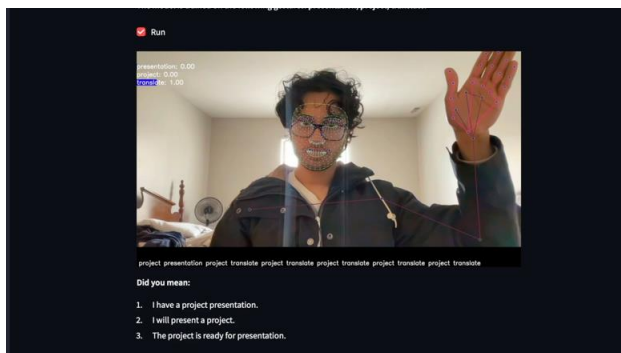


Figure 1: Showing App

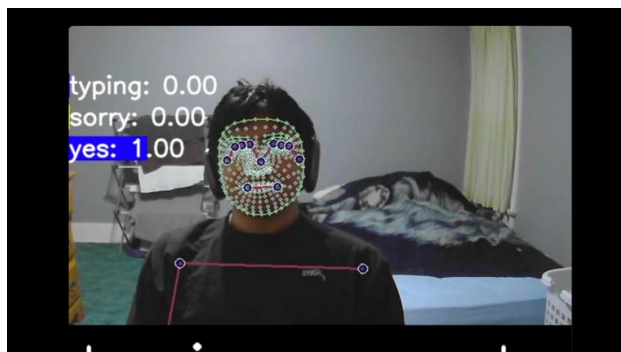


Figure 3: Team member signing typing, sorry, and yes

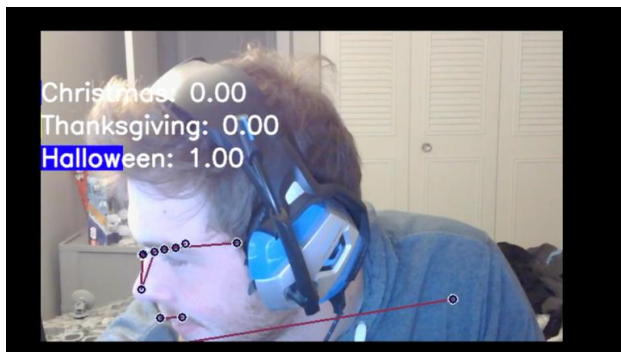


Figure 4: Team member signing Christmas, Thanksgiving, and Halloween



Figure 2: Team member testing symbols learn, read, and write

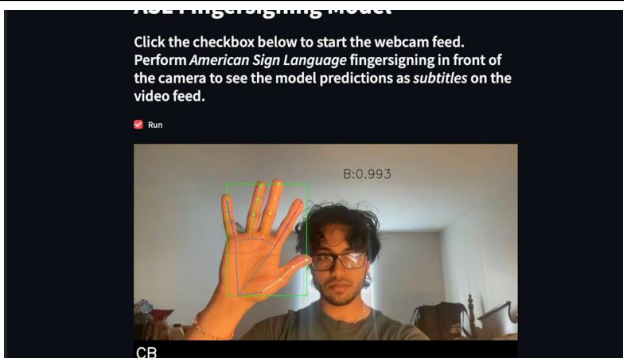
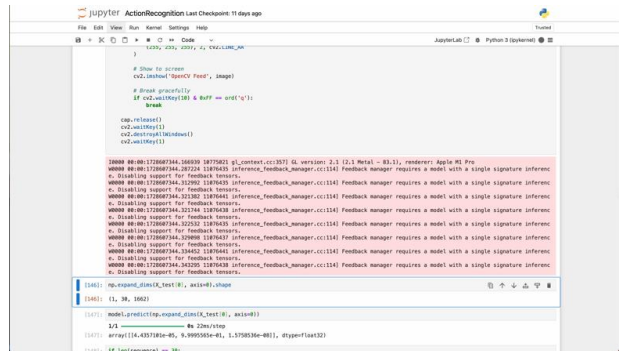
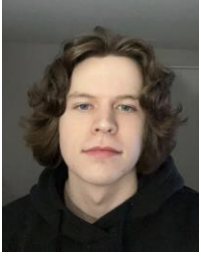


Figure 6: Team member signing letters in fingersigning model

AUTHORS

	Pranaav Iyer will receive a B.S degree in 2025 in Computer Science from Purdue University, Indiana, United States. He is currently working towards the M.S. degree in Artificial Intelligence. He is currently seeking jobs relating to ML/AI and full stack development. His research interests lie in Artificial Intelligence and Human-Machine Interaction.
	Michael Nguyen will receive a B.S degree in 2025 in Computer Science from Purdue University, Indiana, United States. He is currently seeking full time positions relating to software development and full stack development. His research interests lie in full stack development and software engineering.
	Oscar Primitivo is set to earn his Bachelor of Science in Computer Science from Purdue University, Indiana, in 2025. His interest lie in full-stack development, with a focus on front-end technologies. He is currently looking for full-time software positions
	Nick Everett is set to receive his Bachelor of Science degree in computer science in spring 2025 from Purdue University, Indiana, United States. He is currently seeking a full-time position relating to full stack engineering with a focus on front end development.
	Griffin Collins is set to receive his Bachelor of Science degree in Computer Science in spring 2025 from Purdue University, Indiana, United States. He is currently seeking full-time opportunities relating to Software and Full-Stack engineering.